

Week	Practical AIM
Week 1	Title: Introduction to React and Component Architecture Objective: To set up a React development environment and build a static UI using reusable components. Problem Statement: Create a React application for a student portfolio page. The page must include at least 4 reusable components (Header, About, Skills, Footer). Each component must be independently structured and composed into a single-page layout.
Week 2	Title: State Management and Routing in React Objective: To implement reactive state management and multi-page navigation using React Router. Problem Statement: Extend the portfolio application from Practical 1 by adding a navigation bar with at least 3 routes (Home, Projects, Contact). Implement state to dynamically show/hide content and manage a simple form input on the Contact page.
Week 3	Title: API Integration and Data Rendering in React Objective: To consume a REST API in React and handle asynchronous data with loading and error states. Problem Statement: Integrate a public REST API (e.g., GitHub API) into the portfolio application to fetch and display a list of repositories dynamically. Implement loading spinner and error message components to handle all possible response states.
Week 4	Title: Building a RESTful API with Node.js and Express Objective: To design and implement a RESTful backend server with complete CRUD endpoints using Express middleware pipeline. Problem Statement: Build a Node/Express backend for a Task Management system. Implement REST endpoints for creating, reading, updating, and deleting tasks. Apply request logging middleware and a global error handling middleware to the pipeline.
Week 5	Title: MongoDB Integration and Schema Design with Mongoose Objective: To connect a MongoDB database to an Express server and enforce data validation through Mongoose schema. Problem Statement: Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose. Define a Task schema with appropriate field types, required constraints, and default values. Test all CRUD operations against the live database.
Week 6	Title: Full Stack Integration – React + Node + MongoDB Objective: To wire the React frontend to the Node/Express/MongoDB backend into a fully functional full-stack application. Problem Statement: Connect the React portfolio/task UI to the Node+MongoDB backend built in Practicals 4 and 5. The application must support creating, viewing, updating, and deleting tasks from the React interface through API calls, with all data persisted in MongoDB.
Week 7	Title: Authentication and Middleware Pipeline Objective: To implement JWT-based authentication and input validation as part of the Express middleware pipeline. Problem Statement: Add user registration and login functionality to the Task Management application. Implement JWT token generation on login, protect task routes using an authentication middleware, and add server-side input validation to reject malformed requests before they reach the database.
Week 8	Title: Performance Optimization and Lazy Loading in React Objective: To improve frontend performance using lazy loading and code splitting techniques. Problem Statement: Optimize the React frontend of the Task Management application by implementing lazy loading for route-based components using React.lazy() and Suspense. Measure and compare bundle size and load time before and after optimization using browser developer tools.
Week 9	Title: In-Memory Caching and Query Optimization Objective: To implement server-side caching and measure its impact on API response time. Problem Statement: Add in-memory caching to the Task Management backend using node-cache. Cache the GET all tasks response and invalidate the cache on any write operation. Record and compare API response times with and without caching using a tool like Postman or Thunder Client.
Week 10	Title: Asynchronous Processing with Event-Driven Architecture Objective: To implement asynchronous background processing using Node.js native EventEmitter without external dependencies. Problem Statement: Add a background notification system to the Task Management application using Node.js built-in EventEmitter. When a new task is created via the API, emit a task-created event that is handled asynchronously to log a notification (e.g., task title, timestamp, assigned user). The main API response must return immediately without waiting for the background process to complete.
Week 11	Title: Containerization with Docker and Docker Compose Objective: To containerize the full-stack application using Docker and orchestrate services using Docker Compose. Problem Statement: Write Dockerfiles for both the React frontend and Node/Express backend of the Task Management application. Create a Docker Compose file that orchestrates the frontend, backend, and MongoDB containers together. The entire application must run with a single docker-compose up command.
Week 12	Title: CI/CD Pipeline with GitHub Actions Objective: To automate testing and deployment of the application using a CI/CD pipeline. Problem Statement: Set up a GitHub Actions workflow for the Task Management application that automatically runs on every push to the main branch. The pipeline must install dependencies, run at least one automated test, and report pass/fail status. Demonstrate a failed pipeline caused by a broken test and a successful pipeline after the fix.
Week 13	Title: AI API Integration into a Web Application Objective: To integrate an external AI service into the existing application and understand responsible AI usage. Problem Statement: Add an AI-powered feature to the Task Management application using the OpenAI API (or an alternative such as Gemini API). The feature must serve a meaningful purpose – for example, auto-generating task descriptions, prioritizing tasks, or suggesting deadlines based on task title. Handle API errors gracefully and ensure the application remains functional if the AI service is unavailable.